Load balancer w Go — wskazówki do zadania rekrutacyjnego

Najważniejsze: potraktuj zadanie nie jako „napisz pętlę round-robin”, tylko jako zaprojektowanie małego, poprawnego **HTTP reverse proxy**. Na poziomie seniora będą oceniane głównie: współbieżność, obsługa błędów, timeouty, health checki, graceful shutdown, testowalność i świadome ograniczenia.

Standardowa biblioteka Go ma już `httputil.ReverseProxy`, więc nie ma sensu ręcznie kopiować requestów i response’ów, chyba że zadanie wyraźnie wymaga implementacji proxy od zera.

Zacznij od założeń

W README albo na początku rozwiązania napisz jawnie:

- HTTP, nie TCP/L4.
- Load balancing po każdym requestcie, nie po połączeniu TCP.
- Algorytm: round-robin jako wersja podstawowa.
- Backend może być chwilowo niedostępny.
- Health check: np. `GET /healthz`.
- Request timeout i transport z connection poolingiem.
- Brak retry dla metod nieidempotentnych, np. `POST`, chyba że specyfikacja mówi inaczej.
- Konfiguracja przez flagi lub plik.
- Graceful shutdown po `SIGTERM`/`SIGINT`.

To pokazuje, że rozumiesz różnicę między implementacją a produkcyjnym zachowaniem.

Proponowana architektura

Podziel projekt na kilka komponentów:

```
cmd/lb/main.go
internal/config/config.go
internal/balancer/pool.go
internal/balancer/backend.go
internal/proxy/proxy.go
internal/health/checker.go
```

Minimalny model:

```
type Backend struct {
    URL    *url.URL
    Proxy  *httputil.ReverseProxy
```

```
    Healthy atomic.Bool
}

type Pool struct {
    backends atomic.Value // [] *Backend
    next     atomic.Uint64
}
```

`atomic.Value` jest przydatne, jeżeli chcesz bezpiecznie podmieniać całą listę backendów podczas health checka lub reloadu konfiguracji. Jeśli lista jest stała, wystarczy immutable slice i atomowy indeks.

Round-robin

Nie używaj zwykłego:

```
pool.next++
```

Handler HTTP działa współbieżnie, więc wystąpi race condition. Użyj atomika:

```
func (p *Pool) Next() *Backend {
    backends := p.backends.Load().([]*Backend)
    if len(backends) == 0 {
        return nil
    }

    start := p.next.Add(1)

    for i := uint64(0); i < uint64(len(backends)); i++ {
        candidate := backends[(start+i)%uint64(len(backends))]
        if candidate.Healthy.Load() {
            return candidate
        }
    }

    return nil
}
```

Warto iterować po wszystkich backendach, a nie tylko po jednym. Jeżeli backend wybrany przez round-robin jest niedostępny, następny zdrowy backend powinien nadal dostać request.

Uważaj na pustą listę — modulo przez zero byłoby błędem.

Reverse proxy

Dla każdego backendu utwórz proxy raz, a nie przy każdym requestcie:

```
func newBackend(rawURL string) (*Backend, error) {
    target, err := url.Parse(rawURL)
    if err != nil {
        return nil, err
```

```
    }

    proxy := httputil.NewSingleHostReverseProxy(target)

    originalDirector := proxy.Director
    proxy.Director = func(req *http.Request) {
        originalDirector(req)

        req.Header.Set("X-Forwarded-Host", req.Host)
        req.Header.Set("X-Forwarded-Proto", "http")
    }

    proxy.ErrorHandler = func(w http.ResponseWriter, r *http.Request, err error) {
        http.Error(w, "upstream unavailable", http.StatusBadGateway)
    }

    return &Backend{
        URL:   target,
        Proxy: proxy,
    }, nil
}
```

Lepsza wersja używa `Rewrite` i `ProxyRequest`, szczególnie jeśli chcesz świadomie kontrolować nagłówki. Istotne jest, żeby nie ufać bezrefleksyjnie nagłówkom `X-Forwarded-*` pochodzącym od klienta.

Handler:

```
func (s *Server) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    backend := s.pool.Next()
    if backend == nil {
        http.Error(w, "no healthy backends", http.StatusServiceUnavailable)
        return
    }

    backend.Proxy.ServeHTTP(w, r)
}
```

Timeouty i transport

Nie twórz `http.Client` w każdym żądaniu. Użyj współdzielonego `http.Transport`:

```
transport := &http.Transport{
    Proxy: http.ProxyFromEnvironment,

    DialContext: (&net.Dialer{
        Timeout:   2 * time.Second,
        KeepAlive: 30 * time.Second,
    }).DialContext,

    MaxIdleConns:        100,
    MaxIdleConnsPerHost: 20,
    MaxConnsPerHost:     100,
```

```
    IdleConnTimeout:     90 * time.Second,

    TLSHandshakeTimeout:   2 * time.Second,
    ResponseHeaderTimeout: 5 * time.Second,
    ExpectContinueTimeout: 1 * time.Second,
}
```

Następnie przypisz transport do proxy:

```
proxy.Transport = transport
```

Ważne rozróżnienie:

- `DialContext` timeout — jak długo czekamy na nawiązanie połączenia.
- `ResponseHeaderTimeout` — jak długo czekamy na nagłówki backendu.
- `IdleConnTimeout` — jak długo trzymamy bezczynne połączenie.
- `context.Context` requestu — pełny deadline żądania, jeśli jest wymagany.

Nie ustawiaj bezmyślnie jednego globalnego timeoutu, który może przerwać długie, legalne odpowiedzi.

Health check

Najprostszy checker powinien:

1. cyklicznie odwiedzać każdy backend,
2. używać krótkiego timeoutu,
3. oznaczać backend jako zdrowy lub niezdrowy,
4. nie blokować obsługi requestów.

```
func checkBackend(ctx context.Context, client *http.Client, b *Backend) bool {
    req, err := http.NewRequestWithContext(
        ctx,
        http.MethodGet,
        b.URL.ResolveReference(&url.URL{Path: "/healthz"}).String(),
        nil,
    )
    if err != nil {
        return false
    }

    resp, err := client.Do(req)
    if err != nil {
        return false
    }
    defer resp.Body.Close()

    _, _ = io.Copy(io.Discard, io.LimitReader(resp.Body, 4<<10))
```

```
    return resp.StatusCode >= 200 && resp.StatusCode < 300
}
```

Nie oznaczaj backendu jako dead po jednym przypadkowym błędzie. Lepszy model:

- `failureThreshold = 3`,
- `successThreshold = 2`,
- osobne liczniki sukcesów i porażek,
- stan `healthy/unhealthy`.

W zadaniu rekrutacyjnym możesz zacząć od prostszego rozwiązania, ale opisz, że produkcyjnie dodałbyś hysteresis, żeby uniknąć flappingu.

Checker powinien być zamykany przez kontekst:

```
func (c *Checker) Run(ctx context.Context) {
    ticker := time.NewTicker(c.interval)
    defer ticker.Stop()

    c.checkAll(ctx)

    for {
        select {
        case <-ticker.C:
            c.checkAll(ctx)
        case <-ctx.Done():
            return
        }
    }
}
```

Nie twórz osobnej goroutine bez kontroli dla każdego ticka, bo przy wolnych health checkach możesz nakładać na siebie kolejne rundy.

Błędy i retry

To często miejsce, gdzie kandydaci popełniają błąd.

Bezpieczne domyślne zachowanie:

- brak backendu: `503 Service Unavailable`,
- błąd połączenia z backendem: `502 Bad Gateway`,
- timeout backendu: zazwyczaj `504 Gateway Timeout`,
- retry tylko przy błędzie przed wysłaniem requestu do upstreamu,
- retry przede wszystkim dla `GET`, `HEAD`, czasem `OPTIONS`,
- nie retryuj automatycznie `POST`, `PUT` ani `PATCH`.

Retry musi mieć limit:

```
const maxAttempts = 2
```

I powinien omijać backend, który właśnie zawiódł. Nie implementuj retry „w ciemno” w `ErrorHandler`, bo możesz nie wiedzieć, czy odpowiedź częściowo trafiła już do klienta.

W prostym zadaniu lepiej jasno napisać:

> This implementation does not retry requests to avoid duplicating side effects. Failed requests return 502/504.

To jest bardziej dojrzałe niż niebezpieczny retry.

Graceful shutdown

Przykładowy schemat:

```
server := &http.Server{
    Addr:              ":8080",
    Handler:           lb,
    ReadTimeout:       5 * time.Second,
    ReadHeaderTimeout: 5 * time.Second,
    WriteTimeout:      30 * time.Second,
    IdleTimeout:       60 * time.Second,
}

ctx, stop := signal.NotifyContext(
    context.Background(),
    syscall.SIGINT,
    syscall.SIGTERM,
)
defer stop()

go func() {
    <-ctx.Done()

    shutdownCtx, cancel := context.WithTimeout(
        context.Background(),
        10*time.Second,
    )
    defer cancel()

    _ = server.Shutdown(shutdownCtx)
}()
```

Warto omówić:

- nowe requesty nie powinny być przyjmowane po rozpoczęciu shutdownu,
- trwające requesty powinny dostać czas na zakończenie,
- health checker powinien zostać zatrzymany,
- transport powinien zamknąć idle connections.

Testy, które warto napisać

Użyj `httptest.Server`, nie prawdziwych procesów backendów.

Minimum:

- round-robin rozdziela requesty po równo,
- request przy pustej puli zwraca `503`,
- niezdrowy backend jest pomijany,
- backend wraca po ponownym health checku,
- błąd backendu zwraca `502`,
- timeout backendu nie zawiesza load balancera,
- równoległe requesty nie powodują race condition,
- graceful shutdown kończy pracę,
- nagłówki i ścieżka są przekazywane poprawnie.

Uruchom:

```
go test ./...
go test -race ./...
go vet ./...
go test -run TestLoadBalancer -count=50 ./...
```

`-race` jest szczególnie ważny, bo `http.Server` wykonuje handlery współbieżnie.

Pytania, które mogą paść

Dlaczego `ReverseProxy`, a nie `http.Client`?

`ReverseProxy` prawidłowo obsługuje strumieniowanie requestu i response'u, nagłówki hop-by-hop oraz kopiowanie danych. Ręczne `io.ReadAll` zwiększa zużycie pamięci i psuje streaming dużych odpowiedzi.

Round-robin czy least-connections?

Round-robin jest prosty i tani, ale zakłada podobny koszt requestów. Least-connections może lepiej działać przy długich requestach, ale wymaga poprawnego, współbieżnego licznika i nadal nie mierzy realnego czasu CPU ani obciążenia backendu.

Co się stanie, jeśli backend padnie po health checku?

Health check nie gwarantuje dostępności w momencie obsługi requestu. Dlatego proxy musi obsługiwać błędy runtime, a health check tylko ogranicza prawdopodobieństwo kierowania ruchu do niedostępnego backendu.

Czy worker pool jest potrzebny?

Dla HTTP reverse proxy zwykle nie. `net/http` już obsługuje requesty współbieżnie, a ograniczenie powinno być realizowane przez limity połączeń, semafor lub limit per backend. Worker pool może sztucznie zwiększyć opóźnienia, jeśli zadania są blokujące.

Co z WebSocketami?

`httputil.ReverseProxy` może obsługiwać upgrade'y, ale trzeba przetestować zachowanie oraz timeouty. Przy klasycznym zadaniu HTTP możesz jawnie zaznaczyć, że WebSocket nie jest w zakresie.

Sensowna kolejność implementacji

1. Jeden backend i działający reverse proxy.
2. Lista backendów.
3. Round-robin z atomowym indeksem.
4. Pomijanie niezdrowych backendów.
5. Health checker.
6. Transport i timeouty.
7. Graceful shutdown.
8. Testy równoległe i `go test -race`.
9. Metryki i dokumentacja ograniczeń.

Nie zaczynaj od metryk, retry i dynamicznej konfiguracji. Najpierw doprowadź do poprawnego przepływu requestu.

Co dodać, jeśli zostanie czas

- Prometheus metrics: request count, latency, status codes, backend errors.
- `Connection: close` i nagłówki hop-by-hop.
- Weighted round-robin.
- Least-connections.
- Dynamiczne dodawanie/usuwanie backendów.
- Circuit breaker.
- Konfiguracja przez YAML/JSON.
- Endpoint `/ready` i `/live`.
- Structured logging.
- Limity body requestu.
- Ochrona przed zbyt dużą liczbą równoległych requestów.

Najlepsze rozwiązanie rekrutacyjne nie musi mieć wszystkich tych funkcji. Powinno być małe, idiomatyczne, dobrze przetestowane i jasno opisywać, czego celowo nie implementuje.